

# Useful questions; provably few tests

**Question:** Can we design a method that works for every secret, and prove no shorter method can always work? This is decision-tree complexity with exact-position feedback.

## Prepare and check the rules

Use two kinds of counters (mark R/B or distinct symbols), paper position strips, and three folded-paper screens. Use 12 counters of each kind for K and middle, and 16 of each kind for upper so six candidate rows and a test can remain visible. Keep all records. A parent may write dictated guesses and scores. First score visible secret RBR against RRR (2), BRB (0), and RBR (3). Repeated colors are allowed. Give only the **total correct positions**; no wrong-place-color feedback.

Start with page 1 for each child (seven student sheets). Keep the remaining pages as continuation masters, including any earlier records children will compare. Print the extra only when selected. K: no reading; match positions and count 0–2; eliminate impossible examples. Middle: counts to 3, optional R/B reading; compare one changed feature and complete cases. Upper: counts to 4; worst-case reasoning, symmetries, and branching. Extra: 0/1 notation and small equations; variables can be replaced by counters.

## Count the same thing throughout

A **test** is a scored query. Once exactly one secret remains, naming it is free in this investigation. Original game rules require actually submitting the secret, potentially one extra turn. Our exact optima for 2, 3, 4 positions are **2, 3, 4 tests to identify**, not claims of those same turn limits for winning submission. Stop early if a score already identifies the secret.

## A shared hour with seven children and two adults

0–10: explore materials freely. 10–15: brief launch and rules. 15–35: main investigation. 35–40: movement/reset; preserve work. 40–55: continue or choose an optional proof. 55–60: share and tidy. This flexible default is our proposal, not a source prescription.

One adult stays with K,K,1. The organizer alternates middle and upper, leaving a concrete next attempt and returning for an explanation. In games keep two opposing sides and rotate a checker. Proof continuations need adult attention; pages are not a completion quota.

**K-1 stop:** Build a complete two-test method. **Optional proof:** for any first test, build two secrets scoring one, so one test cannot always work.

**Middle stop:** Explain why the three-test method works. **Optional proof:** combine the one-B triple obstruction with color renaming to cover every first test. Without renaming, claim only the RRR-first obstruction.

**Upper stop:** Explain why four tests always suffice. **Optional proof:** defend the triple and four-candidate branches separately, then use symmetry to prove optimality. Choose one older group's proof conversation at a time.

# Two and three places: complete proofs

## K-1: elimination becomes a decision tree

RR scores 0 only for BB, 2 only for RR, and 1 for RB or BR. In the last branch test RB: scores 2 or 0 distinguish them. So two tests suffice. One cannot always work: for every first test, two secrets match in exactly one position. Build those counterexamples.

## Middle: a three-test upper bound

Let baseline RRR score  $s$ , the number of Rs in the secret. Flipping one test position R to B changes its contribution from 1 to 0 if the secret there is R, or 0 to 1 if it is B. The new score is  $s - 1$  or  $s + 1$ , never unchanged. Tests BRR and RBR reveal positions 1 and 2. Their number of Rs subtracted from  $s$  determines position 3. This procedure covers all eight secrets.

## Middle: why two tests cannot guarantee success

After RRR scores 2, candidates are BRR, RBR, RRB. The complete second-test table is:

| Test | BRR | RBR | RRB |
|------|-----|-----|-----|
| RRR  | 2   | 2   | 2   |
| RRB  | 1   | 1   | 3   |
| RBR  | 1   | 3   | 1   |
| RBB  | 0   | 2   | 2   |
| BRR  | 3   | 1   | 1   |
| BRB  | 2   | 0   | 2   |
| BBR  | 2   | 2   | 0   |
| BBB  | 1   | 1   | 1   |

Every row repeats a score; some answer leaves two candidates. Conceptually, the three secrets differ only in the position of one B. A test only reveals whether that B falls inside its B positions, giving at most two outcomes. For the numbered symmetry step, physically turn BRB into RRR by swapping R/B at places 1 and 3 in both secrets and queries. An equal pair stays equal and an unequal pair stays unequal. Every score is preserved. Do this at the B positions of any first query; it becomes RRR, so the lower bound covers all first queries and all adaptive methods.

## Hints

Make candidate rows tangible. The table is optional once a child explains the one-B structure and two possible scores; further entries add no proof. Test a troublesome pair first. Ask what score a test would earn for each candidate. A test need not itself be a possible secret to reveal useful information.

# Four tests: upper and lower bounds

## Upper bound

RRRR and three separate single-position flips determine places 1–3 by score changes; the baseline R count determines place 4. Four tests suffice. Generally this uses  $n$  tests to identify an  $n$ -bit secret, at most  $n + 1$  turns if winning submission is required.

## A difficult answer that stays consistent

By independent color renaming, take the first test to be RRRR. Answer 2, leaving six two-B secrets. Classify the second test by its B count. Position permutations and complementing a test (replacing score  $s$  by  $4 - s$ ) reduce all cases:

**0 or 4 Bs:** no information; six remain.

**1 B (BRRR):** scores 3 or 1 each leave three candidates. Take score 3: BBRR, BRBR, BRRB. Ask: *Which position is fixed? What is the only feature that changes?* They share the first B; the other B occupies one of three places. Any last test has a fixed first-position contribution and only two possible scores on the remaining one-B triple. It cannot separate all three. The score-1 branch has the same structure; 3-B tests are complementary.

**2 Bs (BBRR):** score 4 leaves BBRR, score 0 leaves RRBB, score 2 leaves BRBR, BRRB, RBBR, RBRB. Choose the four-candidate branch. For any last test with  $k$  Bs and a two-B secret, let  $j$  positions have B in both. The score is  $2 - k + 2j$ . There are at most three possible scores, so four candidates cannot all be separated.

Every second test has a consistent branch that needs more than one further test. Thus three tests cannot guarantee identification; four is optimal. The lower bound uses actual score structure, not merely five possible answers per test.

## Facilitating a hard proof

Let the child physically sort six code cards into score piles. Ask them to make the largest pile small, then play the stubborn setter. Upper pages 1–2 supply a satisfying stop. Pages 3–4 are an optional adult-supported lower-bound investigation; the final-query triple argument is different from the four-candidate counting argument. If full optimality is too much, obtain a proof of the four-test method and one failed three-test plan. Record that boundary honestly.

**Relationship to Week 3:** Here the hidden object is an ordered binary string, queries may adapt, and the goal is a worst-case measurement optimum. Score structure and adversarial lower bounds deepen earlier identification ideas.

# Five bits and a research landmark

## Extra: decode and prove

The example is **10110**, with scores 2,2,2,0. Let  $w$  count secret 1s and  $s_0, s_1, s_2, s_3$  be scores in printed order. Then  $w = 5 - s_0$ , and

$$x_4 + x_5 = \frac{s_1 - s_0 + 2}{2}, \quad x_3 + x_5 = \frac{s_2 - s_0 + 2}{2}, \quad x_2 + x_5 = \frac{s_3 - s_0 + 2}{2}.$$

Each 0-to-1 change gains one match for a secret 1 and loses one for 0. If any pair-sum is 0 or 2, it fixes  $x_5$ , then  $x_2, x_3, x_4$ ;  $w$  fixes  $x_1$ . Otherwise all three sums are 1. If  $x_5 = 0$ , the middle bits are 111 and  $w$  is 3 or 4. If  $x_5 = 1$ , they are 000 and  $w$  is 1 or 2. The ranges do not overlap. So  $w$  fixes the case and  $x_1$ .

This proves four *preselected* tests distinguish all 32 secrets. The page claims sufficiency, not an independently proved five-place optimum. Matching scores encode binary-hypercube distances; distinguishing landmarks form a resolving set.

## Undergraduate and research connections

Decision trees, adversary arguments, Hamming distance, and information bounds belong to discrete mathematics and algorithm analysis. The small optima here have explicit upper and lower proofs.

Cáceres et al., *On the Metric Dimension of Cartesian Products of Graphs* (2005 preprint; 2007 publication), §2, PDF p. 3, gives exactly our four five-bit landmarks. §6, PDF p. 8, equates exact-position static Mastermind with resolving sets in Hamming graphs. This directly adapts a finite construction from a research paper, with our elementary decoding proof.

Knuth, *The Computer as Master Mind* (1976–77), gives a five-guess strategy for commercial four-place, six-color Mastermind; see Knuth's author page and strategy link. That game also has wrong-place-color feedback. Its result does not directly apply here. The shared idea is minimizing the worst surviving candidate set.

## Teaching lineage and future use

JRMF *Crack the Code*, PDF pp. 2–4 supplies binary exact-position play and the  $n + 1$ -turn challenge. The optimum/adversary tasks and extra deepen it. *Math Circle by the Bay*, pp. ix–x supports manipulatives, varied pace, and optional challenges.

Use F06-K-v3/M-v3/U-v3/X-v3; prepared, not taught. Save more colors, noise, and average-case analysis for later. `plans/verify-week-06.py` searches all adaptive query trees for small optima and checks all 32 extra signatures.

# Make the scores usable first

## Revision v3: unpiloted

The September 27 revision applies the Week 1 classroom guidance; no Week 6 classroom outcomes have been reported. We added repeated visible scoring, contrasting single-position changes, physical candidates, and separate recording space before trees and adversary arguments. These are design hypotheses to test, not claims of observed success.

## A concrete whole-group launch

After handling and copying counters, keep RBR visible. Build RRR alongside it and point to matches in positions 1 and 3. Say only “2” and record “RRR: 2.” Compare BRB (0) and RBR (3). Let a child check a test and dictate the log entry. Hide a secret only when this action is understood. Give first pages to groups; demonstrate branches locally after candidate filtering, never as the opening notation.

K page 1: against RB, tests RR,RB,BR,BB score **1,2,0,1**. Page 2: RR gives **2,1,1,0** against RR,RB,BR,BB. On remaining RB/BR, RR scores (1,1), RB scores (2,0), and BR scores (0,2). Page 3 introduces the tree to reuse those trials. It gives one test for RR/BB and two for RB/BR. The one-test lower bound remains an oral follow-up after testing the tree on all four secrets.

Middle page 1: visible RBR has scores 2,0,3. Page 2’s rows BRR,RBR,RRB,BBB have score triples **(2,3,1), (2,1,3), (2,1,1), (0,1,1)**. Ask what a gain/loss says about the changed position before requesting the general method. A working three-test method is a satisfying stopping point. Page 3 uses a small selection of tests plus children’s own before the optional full-table argument. In Problem 6, swapping positions 1 and 3 sends RRB,BBB,BRR to BRR,RBR,RRB; every old and new score is 2. Checking examples motivates the symmetry proof on page 2 of this guide; examples alone do not prove that all first tests are covered.

Upper page 1: score pairs for RBRB,BRBR,RRRR,BBBB are **(2,1), (2,3), (4,3), (0,1)**. Upper page 2’s four-test records for RBRB,BRBR,RBBB,RRRR are **(2,1,3,1), (2,3,1,3), (1,0,2,2), (4,3,3,3)**. Keep each comparison anchored to RRRR. Naming is free throughout.

## Plan the adult conversations

Use 0–10 handling, 10–15 shared launch, 15–35 play/comparison, 35–40 movement, 40–55 current work or one continuation, 55–60 sharing. Starting-page print plan: seven sheets plus continuation masters. K may stop after consistent filtering; middle/upper may stop with a reliable method and its explanation. Return earlier score records alongside later pages. Do not require every table row if the child can already explain its structure.

Observe whether children score positions consistently, compare with the correct baseline, and use records to choose a next test. Record actual examples and claims established, not merely pages distributed. Distinguish observed difficulties, hypotheses about them, and proposed changes. The Week 1 review and Rozhkovskaya’s Lessons 3,7,8 motivate this order; the exact tasks are our adaptations.

# New tables and proof handoffs

## Upper pages 3–4: checked score records

In candidate order BBRR,BRBR,BRRB,RBBR,RBRB,RRBB, second-test rows are:

|      |   |   |   |   |   |   |
|------|---|---|---|---|---|---|
| BBRR | 3 | 3 | 3 | 1 | 1 | 1 |
| BRBR | 4 | 2 | 2 | 2 | 2 | 0 |
| BRRB | 3 | 3 | 1 | 3 | 1 | 1 |
| RBBR | 2 | 2 | 2 | 2 | 2 | 2 |

For the triple BBRR,BRBR,BRRB in Problem 7, final tests RRRR,RBRR,RBBR give **(2,2,2), (3,1,1), (2,2,0)**. For the quartet BRBR,BRRB,RBBR,RBRB in Problem 8, RRRR,BRRR,BBRR give **(2,2,2,2), (3,3,1,1), (2,2,2,2)**. Children's added tests vary; check by counting matches directly.

After these trials, offer the complete lower-bound conversation on page 3 of this guide. A finite collection of failed final tests is not enough. Establish separately why the moving-B triple allows at most two scores and why a two-B quartet allows at most three. Then cover every second test by B-count/position symmetry, and every first test by independent color renaming. Without the last step, report only an RRRR-first obstruction. Keep this distinction in the use log.

## Extra: three pages before the general decoding argument

Page 1 keeps all four probes visible through three examples and two actual games. Probe order is 00000, 00011, 00101, 01001. The provided secrets have records:

|       |   |   |   |   |
|-------|---|---|---|---|
| 00000 | 5 | 3 | 3 | 3 |
| 11111 | 0 | 2 | 2 | 2 |
| 10010 | 3 | 3 | 1 | 1 |

Page 2 first isolates two changed positions. Secret pairs 00,01,10,11 have 00/11 match counts (2,0),(1,1),(1,1),(0,2); changes are **-2,0,0,+2**. Thus score change reveals pair sums 0,1,1,2. The decoding example has total 1-count 3 and pair counts **1,1,0**; it gives 10110 as before. Keep physical counter pairs available before writing equations.

Page 3 gives the ambiguous pair-count case its own examples. If all three pair sums are 1, the four words are **01110,11110,00001,10001**, with total 1-counts **3,4,1,2**. Their four-score records are respectively (2,2,2,2), (1,1,1,1), (4,4,4,4), (3,3,3,3). Any one score distinguishes these four, though these are not all 32 possible secrets. Revisit the hidden games using the counts; then offer the general two-case proof from page 4 of this guide.

Keep the research conclusion precise: four fixed probes suffice to distinguish all 32 five-bit words; this packet does not prove a five-place optimum. The hypercube/landmark translation belongs in the discussion after decoding works. Binary distance is number of mismatches, hence five minus score.